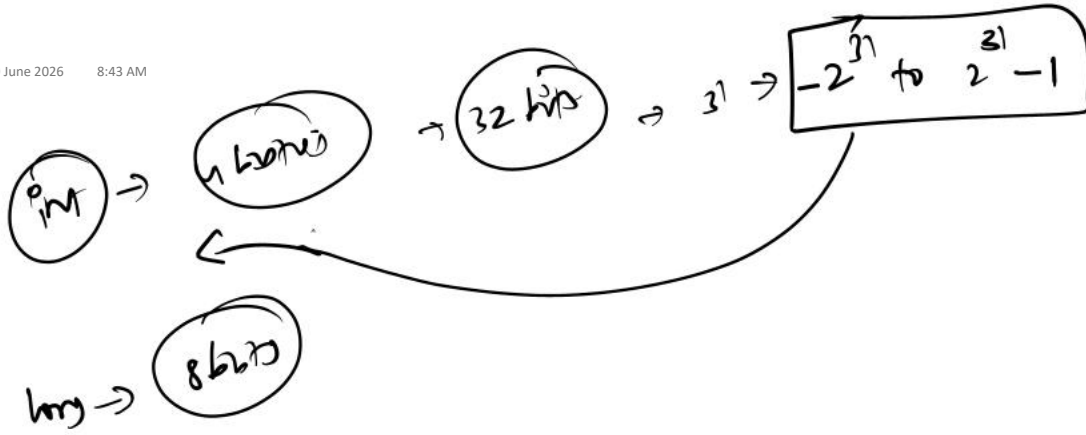
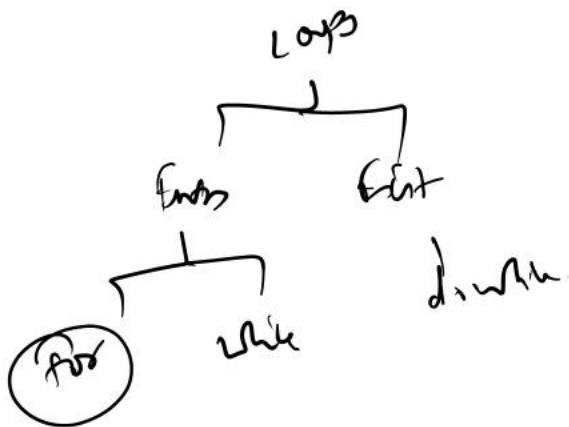


Day 1

Wednesday, 10 June 2026 8:43 AM



int
float
str
bool



```
for (int i = 0; i < 5; i++) {
```

```
    //  
    //
```

```
}
```

for i in range(n):

for i in range(2, n):

|||

exclusive
end

for i in range(2, n, 2):

spaces
inclusive

n(4)

```

- - - *
- - * *
- * * *
* * * *
    
```

n(3)

```

- - *
- * *
* * *
    
```

- ① No of rows. $\rightarrow n$
- ② No of cols in ith row. $\rightarrow n$
- ③ What needs to be printed? $\rightarrow -, *, \text{" "}$

n(4)

```

- - - *  $\rightarrow ①$ 
- - * *  $\rightarrow ②$ 
- * * *  $\rightarrow ③$ 
* * * *  $\rightarrow ④$ 
    
```

i	spaces	stars
1	3	1
2	2	2
3	1	3
4	0	4

stars $\rightarrow$ $\begin{pmatrix} n-1 \\ 1 \end{pmatrix}$

$\begin{pmatrix} n-1 \\ 1 \end{pmatrix}$
space $\rightarrow$

n(4)

```

_ _ _ _ *
_ _ * x x
_ * x x x x
x x x x x x x
_ x x x x x
_ _ x x x
_ _ _ _ *
  
```

n(3)

```

_ _ _ *
_ _ * x x
_ * x x x
_ x x x
_ _ _ *
  
```

n(4)

```

① → _ _ _ _ *
② → _ _ * x x
③ → _ * x x x x
④ → x x x x x x x
⑤ → _ x x x x x
⑥ → _ _ x x x
⑦ → _ _ _ _ *
  
```

first half

i	spaces	stars
1	3	1
2	2	3
3	1	5
4	0	7

second half

n(4)

space → $n-i$

stars → $2i-1$

```

n = int(input("Enter n : "))
# Diamond pattern
# First half
for i in range(1, n + 1):
    # Spaces -> n - i
    for j in range(1, n - i + 1):
        print("_", end = " ")
    # Stars -> 2 * i - 1
    for j in range(1, 2 * i):
        print("*", end = " ")
    print()
# Second half
for i in range(n - 1, 0, -1):
    # Spaces -> n - i
    for j in range(1, n - i + 1):
        print("_", end = " ")
    # Stars -> 2 * i - 1
    for j in range(1, 2 * i):
        print("*", end = " ")
    print()
  
```

print()

n(4)

1	3	5	7
3	5	7	1
5	7	1	3
7	1	3	5

n(5)

1	3	5	7	9
3	5	7	9	1
5	7	9	1	3
7	9	1	3	5
9	1	3	5	7

n(5)

1	3	5	7	9	→ 0
3	5	7	9	1	→ 1
5	7	9	1	3	→ 2
7	9	1	3	5	→ 3
9	1	3	5	7	→ 4

start → $2^i + 1$

end → $2n - 1$

```
n = int(input("Enter n : "))
# Number pattern
end = (2 * n) - 1
for i in range(n) :
    num = (2 * i) + 1
    for j in range(n) :
        print(num, end = " ")
        num += 2
        if num > end :
            num = 1
    print()
```

nums = [10, 20, 30]

ans[0] = 100

0	1	2
10	20	30

@100 @104 @108

(0)

$$ans[i] = B \cdot A + (i \times 4)$$

$$ans(2) = @100 + (2 \times 4)$$

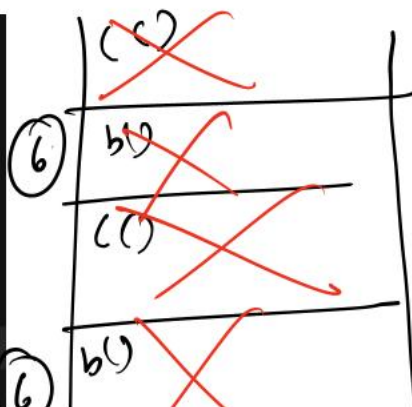
(@100) ans

@104	@108	@112
10	20	30

```

2 def c() :
3     print("Inside c")
4
5 def b() :
6     c()
7     print("Inside b")
8
9 def a() :
10    b()

```



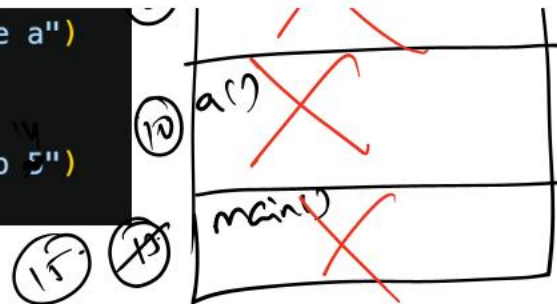
console

Inside c.
Inside b.
Inside a.
End of program

```

11 print("Inside a")
12
13 a()
14 print("In line no 5")
15 b()

```



Inside c
Inside b

1. Two Sum

Solved

Easy Topics Companies Hint

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have **exactly one solution**, and you may not use the same element twice.

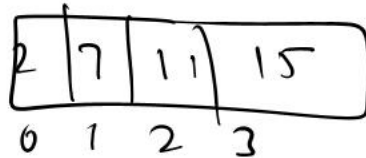
You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

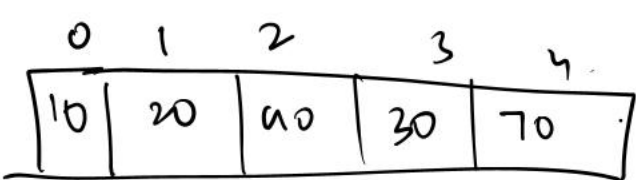
Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.



target 9

`[0,1]` or `[1,0]`



target 100

`[3,4]` → 61p

$i \rightarrow 0 \text{ to } \text{len}(\text{nums}) - 1$
 $j \rightarrow i + 1 \text{ to } \text{len}(\text{nums})$

i	j	nums[i]	nums[j]
0	1	10	20
0	2	10	60
0	3	10	30
0	4	10	70
1	2	20	60
1	3	20	30
1	4	20	70
2	3	60	30
2	4	60	70
3	4	30	70


```

1 class Solution:
2     def twoSum(self, nums: List[int], target: int) -> List[int]:
3         for i in range(len(nums) - 1) :
4             for j in range(i + 1, len(nums)) :
5                 if nums[i] + nums[j] == target :
6                     # output = [j, i]
7                     # return output
8                     return [j, i]

```

↓
p.no: ①

↓
T.C → $O(n^2)$

↓
S.C → $O(1)$

0	1	2	3	4
10	20	40	30	70

10	:	0
20	:	1
40	:	2
30	:	3

100
target

6/8
[4, 3]

i	nums[i]	elem
0	10	90
1	20	80
2	40	60
3	30	70
4	70	30

Add, Get, Remove → $O(1)$ → Dictionary set

```

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {} → creates a dict
        for i in range(len(nums)) :
            elem = target - nums[i]

```

```

if elem in seen :
    j = seen[elem]
    return [j, i]
else :
    seen[nums[i]] = i

```

↓
p.No. (1)

↓
T.C $\rightarrow O(n)$

↓
s.c $\rightarrow O(n)$

217. Contains Duplicate

Solved ✓

Easy

Topics

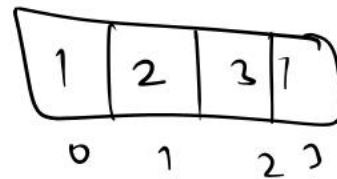
Companies

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`



↓
True

```

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        for i in range(len(nums) - 1):
            for j in range(i + 1, len(nums)):
                if nums[i] == nums[j]:
                    return True
        return False

```

$\rightarrow T.C \rightarrow O(n^2)$

1 sec $\rightarrow 10^4$ operations

2 sec $\rightarrow 10^8$ operations

Two Sum

Contains Duplicate

$$2 \leq n \leq 10^4$$

$$O(n^2) \rightarrow (10^4)^2 \rightarrow 10^8$$

↓
2.5 sec

$$1 \leq n \leq 10^5$$

$$O(n^2) \rightarrow (10^5)^2 \rightarrow 10^{10} \rightarrow 2.5 \text{ sec}$$

$$2.5 > 2 \rightarrow \text{TLE}$$

```
class Solution:
```

```
    def containsDuplicate(self, nums: List[int]) -> bool:
```

```
        seen = set()
```

```
        for i in range(len(nums)) :
```

```
            if nums[i] in seen :
```

```
                return True
```

```
            else :
```

```
                seen.add(nums[i])
```

```
        return False
```

↓
P.No: 217

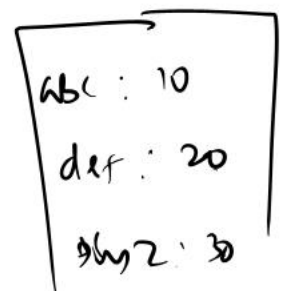
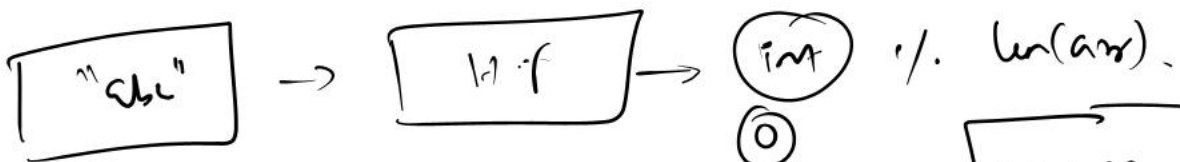
↓
T.C $\rightarrow O(n)$

↓
S.C $\rightarrow O(n)$

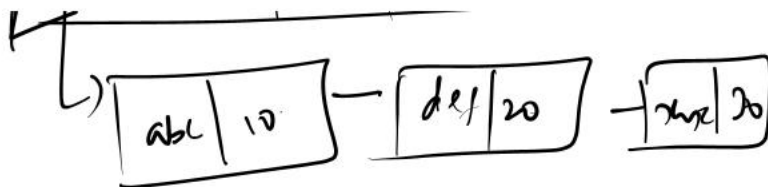
```
class Solution:
```

```
    def containsDuplicate(self, nums: List[int]) -> bool:
```

```
        return len(set(nums)) != len(nums)
```



Answer



Collision

1000

def $\rightarrow$ HT $\rightarrow$ 0

xyz $\rightarrow$ HF $\rightarrow$ 0

Load factor $\rightarrow \frac{n}{b} \leq 0.75$

$\frac{1}{5}$

$\frac{1}{5} \rightarrow 0.2$

$\frac{2}{5} \rightarrow 0.4$

$\frac{3}{5} \rightarrow 0.6$

$\frac{4}{5} \rightarrow 0.8 > 0.75$

Double the size of
BA

$\leftarrow$

Rehashing

11. Container With Most Water

Solved

Medium

Topics

Companies

Hint

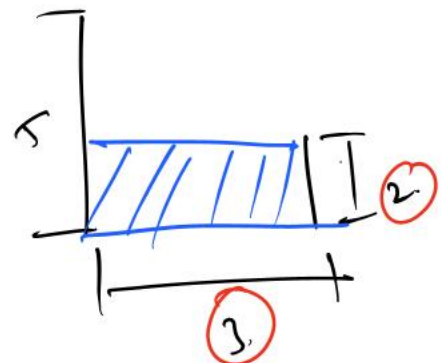
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

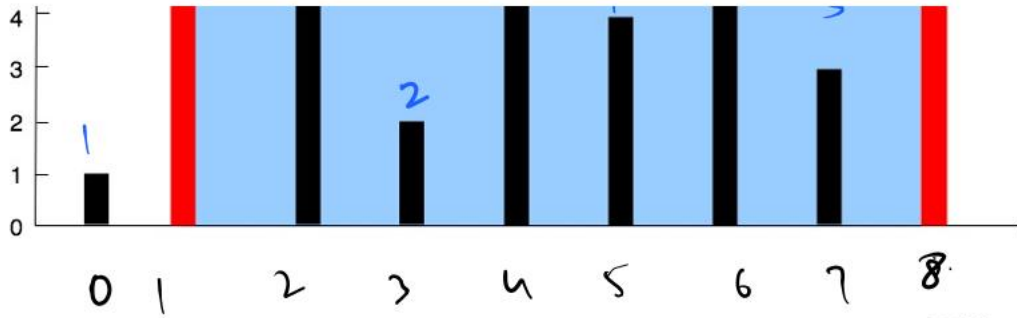
Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Area $= l \times b$



$$l = 8 - 1 = 7.$$

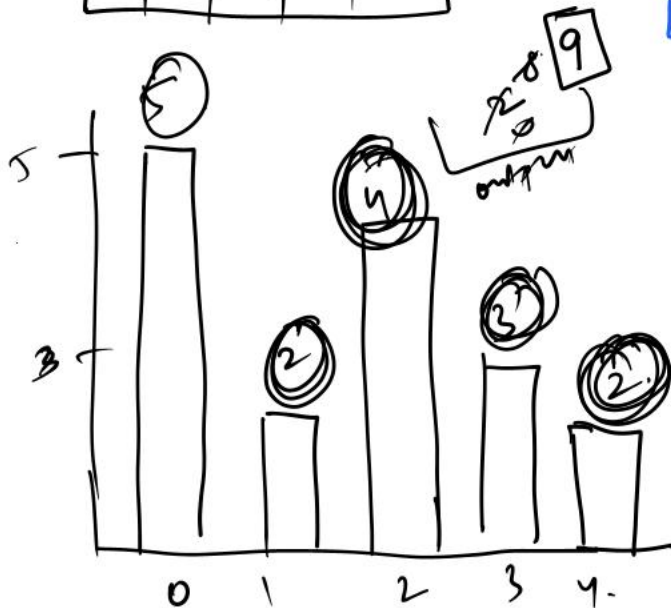


$$b = 7$$

$$a = 7 \times 7 = 49$$

1 8 6 2 5 4 8 3 7

5 2 4 3 2



i	j	len(j-i)	br	area
0	1	1	2	2
0	2	2	4	8
0	3	3	3	9
0	4	4	2	8
1	2	1	2	2
1	3	2	2	4
1	4	3	2	6
2	3	1	3	3
2	4	2	2	4
3	4	1	2	2

9
o/p ↑

i → 0 to n-1
j → i+1 to n

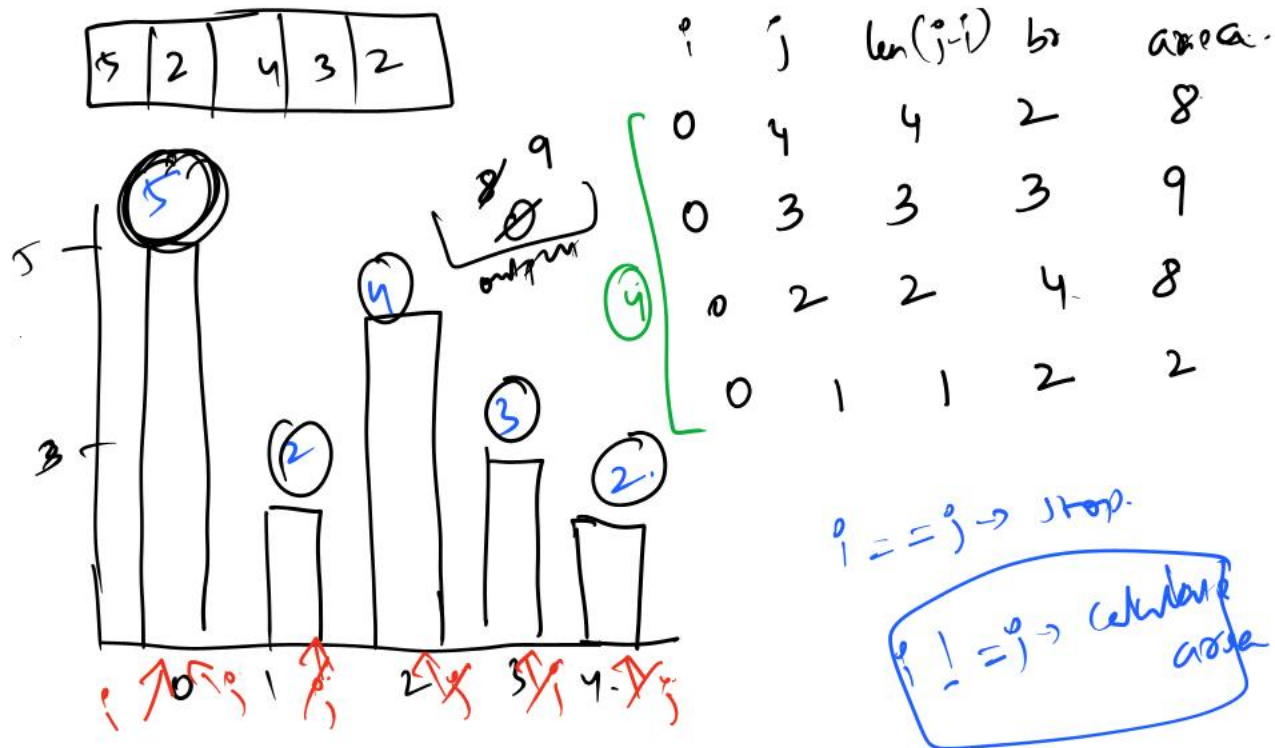
```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        output = 0
        for i in range(len(height) - 1):
            for j in range(i + 1, len(height)):
                length = j - i
                breadth = min(height[i], height[j])
                area = length * breadth
                output = max(area, output)
        return output
```

$$\rightarrow T.C \rightarrow O(n^2)$$

$$2 \leq n \leq 10^5$$

$$(10^5)^2 \rightarrow 10^{10} \rightarrow 2.5 \text{ sec}$$

$$2.5 > 2 \text{ - } \text{TIME}$$



```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        output = 0
        i = 0
        j = len(height) - 1
        while i != j:
            length = j - i
            breadth = min(height[i], height[j])
            area = length * breadth
            output = max(area, output)
            if height[i] < height[j]:
                i += 1
            else:
                j -= 1
        return output
```

$\rightarrow P.N \rightarrow (11)$
 $\rightarrow T.C \rightarrow O(n)$
 $\rightarrow S.C \rightarrow O(1)$

136. Single Number

Solved

Easy Topics Companies Hint

Given a **non-empty** array of integers **nums**, every element appears **twice** except for one. Find that single one.

You must implement a solution with a **linear runtime complexity** and use only **constant extra space**.



4	1	2	1	2
0	1	2	3	4

$2^3 \quad 2^2 \quad 2^1 \quad 2^0$
 $4 \rightarrow \begin{array}{cccc} 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 \end{array} \wedge$
 $\quad \quad \quad 0 \quad 1 \quad 0 \quad 0$
 $1 \rightarrow \begin{array}{cccc} 0 & 0 & 0 & 1 \\ \hline 0 & 1 & 0 & 1 \end{array} \wedge$
 $\quad \quad \quad 0 \quad 1 \quad 0 \quad 1$
 $2 \rightarrow \begin{array}{cccc} 0 & 0 & 1 & 0 \\ \hline 0 & 1 & 1 & 1 \end{array} \wedge$
 $\quad \quad \quad 0 \quad 1 \quad 1 \quad 1$
 $1 \rightarrow \begin{array}{cccc} 0 & 0 & 0 & 1 \\ \hline 0 & 1 & 1 & 0 \end{array} \wedge$
 $\quad \quad \quad 0 \quad 1 \quad 1 \quad 0$
 $2 \rightarrow \begin{array}{cccc} 0 & 0 & 1 & 0 \\ \hline 0 & 1 & 0 & 0 \end{array} \wedge$
 $\quad \quad \quad 0 \quad 1 \quad 0 \quad 0$

$0 \oplus 1$
 $\quad \quad \quad 4$

$$4 \wedge 5$$

$$\begin{array}{r}
 0100 \\
 \oplus 0101 \\
 \hline
 0001 \\
 \downarrow \\
 1
 \end{array}$$

class Solution:

def singleNumber(self, nums: List[int]) -> int:

ans = 0

for i in nums :

ans ^= i

return ans

↓
p.r.o: 136

T.C → $O(n)$

S.C → $O(1)$

121. Best Time to Buy and Sell Stock

Solved

Easy

Topics

Companies

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return the **maximum profit** you can achieve from this transaction. If you cannot achieve any profit, return `0`.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: `5`

5 4 3 2 1

↓
2 → 0/1

10 2 3 1 10 3
0 1 2 3 4 5

output 11 → 0/1

buying price 2 10

i	Buy	sell	profit
0	10	-	-
1	2	-	-
2	2	13	11
3	1	-	-
4	1	10	9
5	1	3	2

class Solution:

```
def maxProfit(self, prices: List[int]) -> int:
    buyingPrice = prices[0]
    output = 0
    for i in range(1, len(prices)) :
        # Buy
        if prices[i] < buyingPrice :
            buyingPrice = prices[i]
        else : # Sell
            profit = prices[i] - buyingPrice
            output = max(output, profit)
    return output
```

→ p.no: 121

→ T.C → $O(n)$

→ S.C → $O(1)$

941. Valid Mountain Array

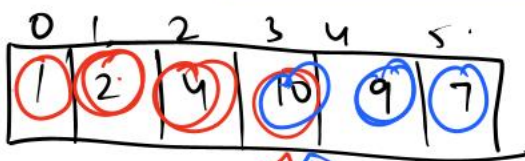
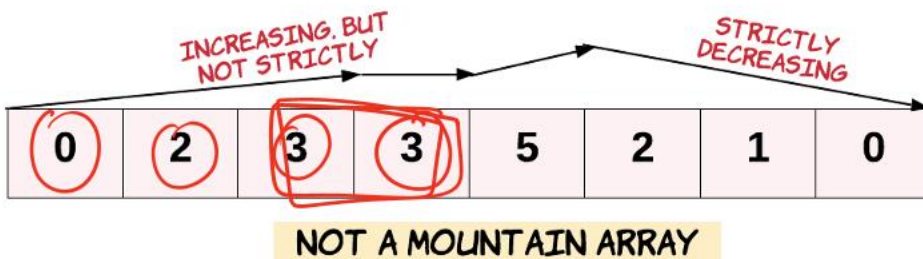
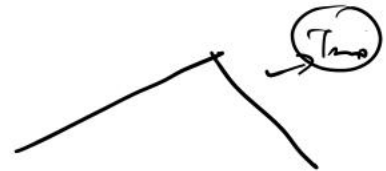
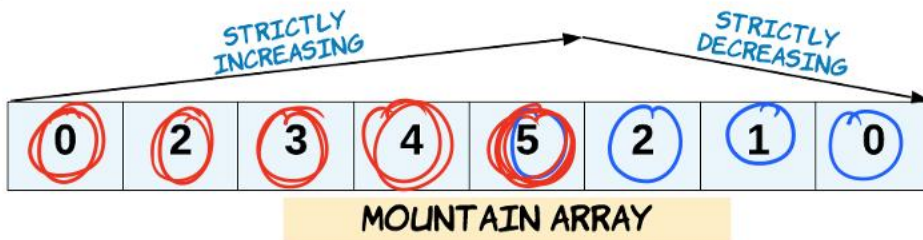
Solved ✓

Easy Topics Companies Hint

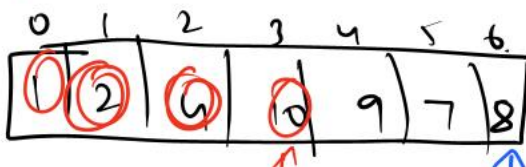
Given an array of integers `arr`, return `true` if and only if it is a valid mountain array.

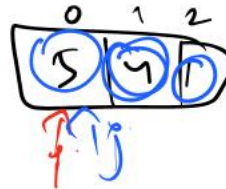
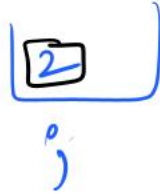
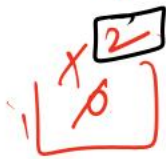
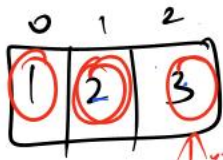
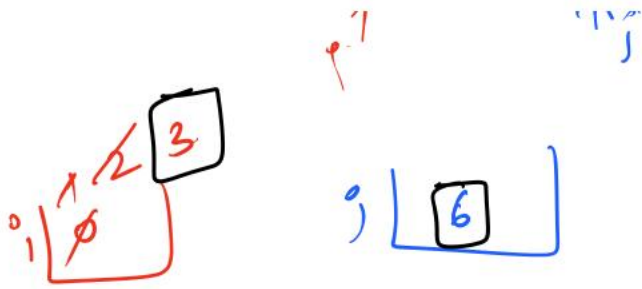
Recall that `arr` is a mountain array if and only if:

- `arr.length >= 3`
- There exists some `i` with $0 < i < arr.length - 1$ such that:
 - `arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`
 - `arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`



Handwritten notes and diagrams illustrating the logic for checking a mountain array. The first part shows a sequence of numbers `1, 2, 4, 10, 9, 7` with arrows indicating the increasing and decreasing parts. The second part shows a sequence of numbers `1, 2, 4, 10, 9, 7` with arrows indicating the increasing and decreasing parts. The third part shows a sequence of numbers `1, 2, 4, 10, 9, 7` with arrows indicating the increasing and decreasing parts.





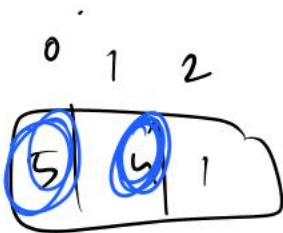
False



$arr(2) \leftarrow arr(3)$

$i < n-1$

$arr[i] < arr[i+1]$



$j > 0$

class Solution:

def validMountainArray(self, arr: List[int]) -> bool:

Strictly increasing

i = 0

while i < len(arr) - 1 and arr[i] < arr[i + 1] :

-> p.No: 941

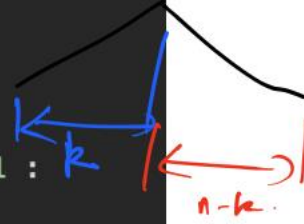
-> T.C -> $O(n)$

```

    i += 1
    # Strictly decreasing
    j = len(arr) - 1
    while j > 0 and arr[j] < arr[j - 1]:
        j -= 1
    if i != j or i == 0 or j == len(arr) - 1:
        return False
    else:
        return True

```

→ s.c → $O(1)$



$k + n - k \rightarrow O(n)$

3637. Trionic Array I

Solved ✓

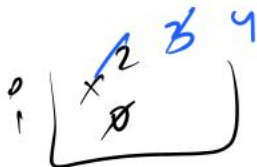
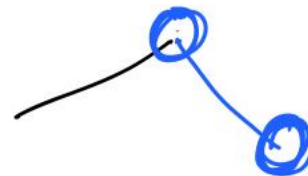
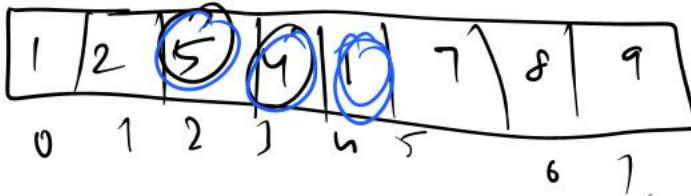
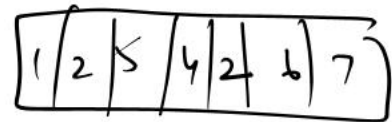
Easy Topics Companies Hint

You are given an integer array `nums` of length `n`.

An array is **trionic** if there exist indices $0 < p < q < n - 1$ such that:

- `nums[0...p]` is **strictly increasing**,
- `nums[p...q]` is **strictly decreasing**,
- `nums[q...n - 1]` is **strictly increasing**.

Return `true` if `nums` is trionic, otherwise return `false`.



```

class Solution:
    def isTrionic(self, nums: List[int]) -> bool:
        n = len(nums)
        i = 0
        # Strictly increasing
        while i < n - 1 and nums[i] < nums[i + 1]:
            i += 1
        if i == 0 or i == n - 1:
            return False
        i = i

```

→ p.no (3637)

→ T.C → $O(n)$

→ S.C → $O(1)$

```

j = 1
# Strictly decreasing
~[ while i < n - 1 and nums[i] > nums[i + 1] :
    i += 1
    if i == j or i == n - 1 :
        return False
# Strictly increasing
~[ while i < n - 1 and nums[i] < nums[i + 1] :
    i += 1
    return i == n - 1

```

$p + q + r \rightarrow \textcircled{n}$